

StockPulse — Complete DevOps Reference Guide

Project: StockPulse (Next.js real-time stock analysis platform) **Repository:** <https://github.com/ItsJupiter000/gitops> **Docker Hub:** jupiter000/stock-pulse **Date:** June 2026

Table of Contents

1. [Concepts Index](#)
 2. [Architecture Overview](#)
 3. [File Structure](#)
 4. [Docker](#)
 5. [Kubernetes \(Minikube\)](#)
 6. [Helm Charts](#)
 7. [ArgoCD & GitOps](#)
 8. [CI/CD Pipeline \(GitHub Actions\)](#)
 9. [Argo Rollouts — Blue-Green Deployment](#)
 10. [NGINX Ingress — Rate Limiting](#)
 11. [Load Balancing](#)
 12. [Daily Startup Commands](#)
 13. [Git Workflow & Common Fixes](#)
 14. [Servers & Ports Reference](#)
 15. [Troubleshooting Guide](#)
 16. [Key Lessons Learned](#)
-

Concepts Index

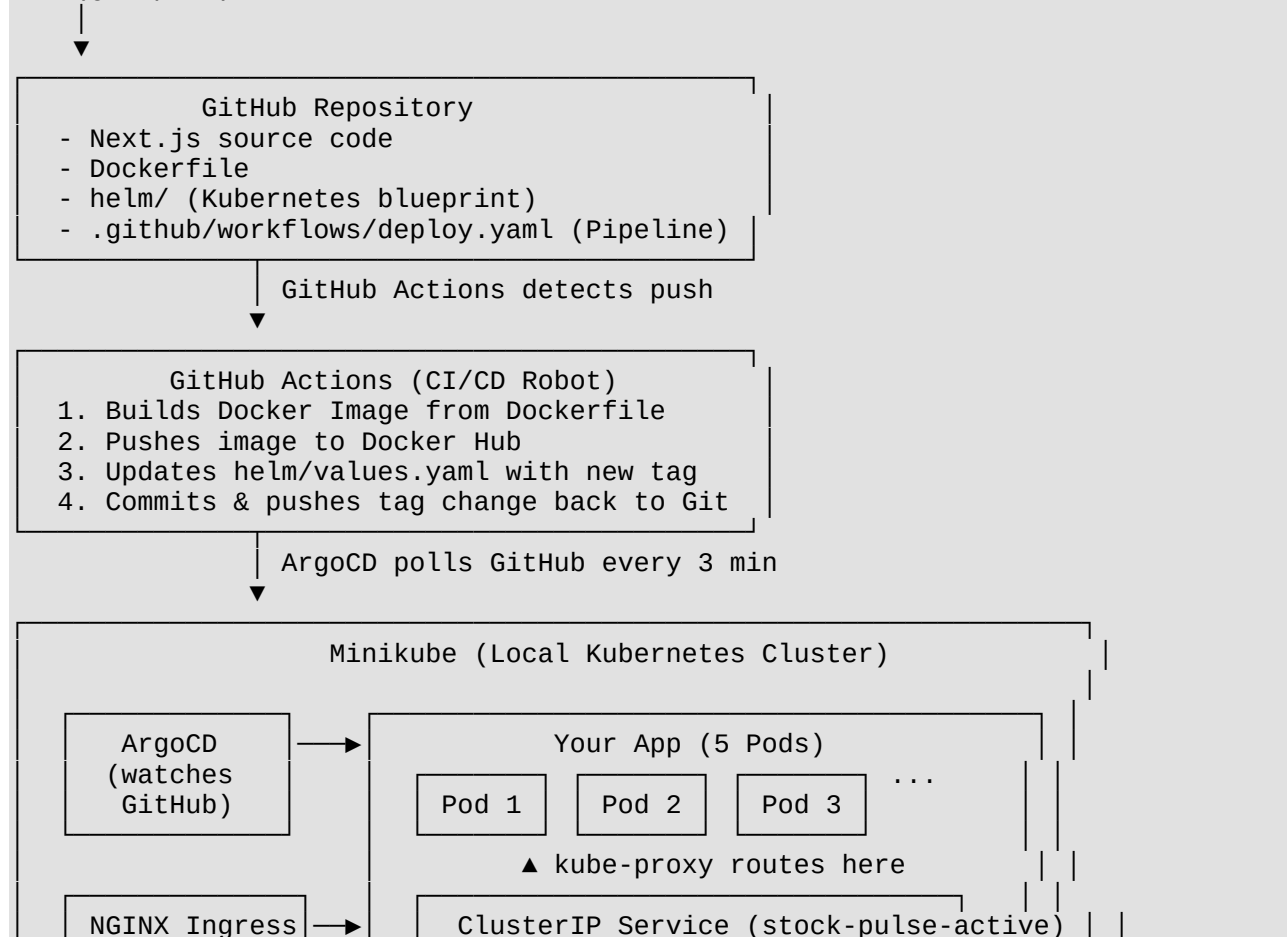
Concept	Section	What it means in plain English
Docker Image	§4	A packaged "box" containing your app and everything it needs to run
Docker Container	§4	A running instance of a Docker image
Multi-stage Build	§4	Smart Dockerfile that builds in a big box but ships in a tiny box

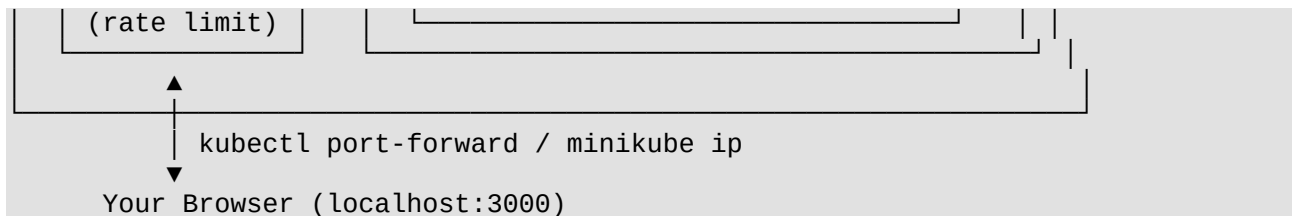
Concept	Section	What it means in plain English
Kubernetes (K8s)	§5	An OS for managing many Docker containers across many servers
Pod	§5	The smallest unit in Kubernetes — one running container
Service (ClusterIP)	§5	An internal load balancer / network address for a group of pods
Ingress	§5	The external gateway that routes internet traffic into the cluster
Minikube	§5	A local, single-machine Kubernetes cluster for development
kubectl	§5	The command-line tool to control Kubernetes (like a steering wheel)
Helm Chart	§6	A reusable blueprint/template for deploying apps to Kubernetes
values.yaml	§6	The config file you edit to customize a Helm chart (replicas, image, etc.)
ArgoCD	§7	A robot that lives in Kubernetes and watches GitHub to auto-deploy
GitOps	§7	A philosophy where Git is the single source of truth for deployments
GitHub Actions	§8	A robot at GitHub that runs your CI/CD pipeline on code push
CI/CD	§8	Continuous Integration / Continuous Deployment — automating builds & deploys
Argo Rollouts	§9	A Kubernetes add-on for advanced deployment strategies
Blue-Green Deployment	§9	Running two versions simultaneously and switching traffic instantly
Promote (Rollouts)	§9	The act of switching live traffic from old version to new version
NGINX Ingress	§10	A powerful web server/proxy

Concept	Section	What it means in plain English
		that sits in front of your cluster
Rate Limiting	§10	Restricting how many requests a single IP can make per second
Load Balancing	§11	Distributing requests evenly across multiple pods
Round Robin	§11	The default load balancing algorithm — pods take turns
kube-proxy	§11	The Kubernetes component that does actual packet routing between pods
Port-Forward	§5	A debugging shortcut to tunnel from localhost to ONE pod — not a load balancer
ClusterIP	§5	A virtual IP for a Service that only exists inside the cluster

Architecture Overview

YOU (git push)





□ File Structure

```
stock-pulse/
├── Dockerfile                                # Recipe to build Docker image (multi-
│                                           stage)
├── .dockerignore                            # Files to exclude from Docker build
context
├── next.config.ts                          # Next.js config (standalone output
enabled)
├── start-system.sh                         # □ Master startup script for the whole
system
├── test-rate-limit.sh                     # □ Script to test NGINX rate limiting
├── test-load-balance.sh                   # □ Script to test Kubernetes load
balancing
├── app/
│   └── api/
│       ├── quote/route.ts                # API: Live price for one stock
│       └── quotes-batch/route.ts         # API: Live prices for all watchlist
stocks
├── chart/route.ts                        # API: Historical OHLCV chart data
├── fundamentals/route.ts                 # API: P/E ratio, Market Cap, Revenue
etc.
├── news/route.ts                        # API: Latest news articles
├── search/route.ts                      # API: Stock symbol autocomplete
├── whoami/route.ts                       # API: Returns pod name (for LB testing)
└── page.tsx                             # Main dashboard page

helm/
├── stock-pulse/
│   ├── Chart.yaml                       # Chart metadata (name, version)
│   └── values.yaml                       # ⚙ Config: replicas, image tag,
ingress rules
├── templates/
│   ├── _helpers.tpl                     # Helper functions for template names
│   ├── rollout.yaml                     # Argo Rollout (Blue-Green strategy)
│   ├── service-active.yaml              # Service: routes traffic to LIVE pods
│   └── service-preview.yaml             # Service: routes traffic to NEW (test)
pods
├── ingress.yaml                          # NGINX Ingress with rate limiting rules

argocd/
├── application.yaml                      # Tells ArgoCD which repo & path to watch

.github/
├── workflows/
│   └── deploy.yaml                       # GitHub Actions CI/CD pipeline
definition
```

□ Docker

Concepts

- **Multi-stage Dockerfile:** The Dockerfile has 3 stages:
 1. **deps** — Installs all npm packages
 2. **builder** — Runs `npm run build` to compile Next.js
 3. **runner** — A tiny Alpine image that only contains the compiled output
- **output: "standalone"** must be set in `next.config.ts` for the slim image to work. This tells Next.js to bundle everything needed into a single folder.

Key Commands

```
# Build image locally to test
docker build -t jupiter000/stock-pulse:test .

# Run it locally on port 3000
docker run -p 3000:3000 jupiter000/stock-pulse:test

# Login to Docker Hub
docker login -u jupiter000

# Push manually to Docker Hub
docker push jupiter000/stock-pulse:latest

# View image layers and sizes
docker history jupiter000/stock-pulse:latest
```

⚙️ Kubernetes (Minikube)

Concepts

- **Pod:** One running container. The smallest unit.
- **ReplicaSet:** Ensures N copies of a Pod are always running.
- **Service:** A virtual IP that load balances traffic across matching pods using labels.
- **Ingress:** The external entry point. Routes incoming requests to the right Service.
- **Namespace:** A way to separate groups of resources. ArgoCD lives in `argocd`, your app lives in `default`.

Installation Commands (One-time)

```
# Install kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/kubectl

# Install Minikube
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-
amd64
```

```
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Start a cluster with 4GB RAM and 2 CPUs using Docker driver
minikube start --driver=docker --memory=4096 --cpus=2

# Enable NGINX Ingress (required for rate limiting)
minikube addons enable ingress
```

Daily Commands

```
# Start Minikube cluster (run every time you restart PC)
minikube start

# Stop cluster (to save resources when done)
minikube stop

# Get the Minikube's IP (for testing via Ingress)
minikube ip

# Check all running pods in default namespace
kubectl get pods

# Check pods in ALL namespaces
kubectl get pods -A

# Check pods + their images (to verify correct version is deployed)
kubectl get pods -l app.kubernetes.io/name=stock-pulse \
  -o jsonpath='{range .items[*]}.{.metadata.name}{"\t"}\n{.spec.containers[0].image}{"\n"}{end}'

# View live console logs from your app pods
kubectl logs -l app.kubernetes.io/name=stock-pulse -f

# Get all services (to see ports and IPs)
kubectl get svc

# Get all resources (pods, services, ingress) at once
kubectl get pods,svc,ingress

# Describe a pod in detail (events, errors, probes)
kubectl describe pod <pod-name>

# Access your app in browser (forward port from cluster to laptop)
kubectl port-forward svc/stock-pulse-active 3000:3000

# Access ArgoCD dashboard in browser
kubectl port-forward svc/argocd-server -n argocd 8080:443

# Scale pods manually (without Helm/ArgoCD)
kubectl scale rollout stock-pulse --replicas=3

# Delete a stuck pod (it will auto-restart)
kubectl delete pod <pod-name>
```

⚠ Important: `kubectl port-forward` is NOT a load balancer!

It creates a persistent TCP tunnel to **one single pod**. All requests go to that one pod. Use it only for debugging. Real load balancing happens through the ClusterIP Service from inside the cluster.

□ Helm Charts

Concepts

- Helm packages Kubernetes YAML files into reusable "charts" with configurable values.
- **Chart.yaml** — Metadata: name, version, description.
- **values.yaml** — Your configuration. Change `replicaCount`, `image.tag`, `ingress` settings here.
- **templates/** — The actual Kubernetes manifests with `{{ .Values.xxx }}` placeholders.

Key values.yaml Settings

```
replicaCount: 5           # How many pods to run (load balancing scale)

image:
  repository: jupiter000/stock-pulse
  tag: "sha-abc123"       # Specific image version (auto-updated by CI/CD)

ingress:
  enabled: true
  className: "nginx"
  annotations:
    nginx.ingress.kubernetes.io/limit-rps: "10"  # Rate limit: requests per
second
    nginx.ingress.kubernetes.io/limit-rpm: "200" # Rate limit: requests per
minute
  hosts:
    - host: stock-pulse.local
```

Key Commands

```
# Lint the chart for syntax errors before pushing
helm lint helm/stock-pulse

# Dry-run to preview what will be deployed without applying
helm template stock-pulse helm/stock-pulse

# Manually install/upgrade the chart (without ArgoCD)
helm upgrade --install stock-pulse ./helm/stock-pulse
```

□ ArgoCD & GitOps

Concepts

- **GitOps:** Git is the single source of truth. Any change to the cluster must go through a Git commit first. Never `kubectl apply` directly in production.
- **ArgoCD** watches your GitHub repository. When it sees a change (like a new image tag in `values.yaml`), it applies the change to Kubernetes automatically.
- **Sync Status:** "Synced" means Kubernetes matches GitHub. "OutOfSync" means a change is

waiting to be applied.

- **App Health:** "Healthy" = all pods running. "Degraded" = pods are failing. "Progressing" = pods are starting up.

Installation (One-time)

```
# Create ArgoCD namespace
kubectl create namespace argocd

# Install ArgoCD
kubectl apply -n argocd -f
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# Wait for all pods to be ready
kubectl get pods -n argocd

# Get the initial admin password
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d; echo
# Username is always: admin
```

Connecting ArgoCD to Your Repository

```
# Apply your ArgoCD Application config
# (This tells ArgoCD WHICH repo to watch and WHERE the Helm chart is)
kubectl apply -f argocd/application.yaml
```

Key Commands

```
# Restart ArgoCD repo-server (fixes "revision must be resolved" errors)
kubectl delete pods -n argocd -l app.kubernetes.io/name=argocd-repo-server

# Force ArgoCD to re-sync immediately from GitHub
kubectl -n argocd app sync stock-pulse

# Check ArgoCD application status from terminal
kubectl -n argocd get applications

# Allow ArgoCD to write back to repository (fixes 403 permission error)
# Go to: GitHub Repo → Settings → Actions → General → Workflow Permissions
# → Select "Read and write permissions" → Save
```

GitHub Secrets Required

Secret Name	Value	Purpose
DOCKERHUB_USERNAME	jupiter000	Login to Docker Hub
DOCKERHUB_TOKEN	(your token)	Password for Docker Hub

□ CI/CD Pipeline (GitHub Actions)

How It Works

Every time you run `git push origin main`:

1. GitHub Actions starts the `deploy.yaml` workflow.
2. **Job 1 (build-and-push):**
 - Checks out your code.
 - Logs into Docker Hub.
 - Builds the Docker image using your `Dockerfile`.
 - Tags it with the Git commit SHA (e.g., `sha-a1b2c3d`).
 - Pushes it to `jupiter000/stock-pulse:sha-a1b2c3d`.
3. **Job 2 (update-helm-chart):**
 - Checks out the repo again.
 - Edits `helm/stock-pulse/values.yaml` — changes `tag: "sha-a1b2c3d"`.
 - Commits this change with message `chore: update image tag to sha-a1b2c3d [skip ci]`.
 - Pushes the commit back to GitHub.
4. ArgoCD sees the new `values.yaml` and deploys the new image.

Why `[skip ci]` in the commit message?

This tag tells GitHub Actions NOT to re-run the pipeline on the automated commit, preventing an infinite loop.

Fix: Workflow Permission Error (403)

If the pipeline fails at "Commit and push changes":

- Go to GitHub Repo → **Settings** → **Actions** → **General**
 - Under "Workflow permissions" → select "**Read and write permissions**"
 - Click **Save**
-

● Argo Rollouts — Blue-Green Deployment

Concepts

- **Blue (Stable/Active):** The current live version all users see.
- **Green (Preview):** The new version running in parallel but not getting user traffic yet.

- **Promote:** The act of switching all traffic from Blue to Green instantly (0ms downtime).
- **Rollback:** Switching traffic back from Green to Blue instantly.
- **autoPromotionEnabled: false** — means the switch is **manual**. You click "Promote" in ArgoCD or run the CLI command. This gives you a chance to test the preview first.

Installation (One-time)

```
# Create namespace and install Argo Rollouts controller
kubectl create namespace argo-rollouts
kubectl apply -n argo-rollouts \
  -f https://github.com/argoproj/argo-rollouts/releases/latest/download/
  install.yaml

# Install the kubectl plugin (for CLI commands)
curl -LO https://github.com/argoproj/argo-rollouts/releases/latest/download/
  kubectl-argo-rollouts-linux-amd64
chmod +x ./kubectl-argo-rollouts-linux-amd64
sudo mv ./kubectl-argo-rollouts-linux-amd64 /usr/local/bin/kubectl-argo-rollouts
```

Key Commands

```
# See the full rollout status (shows Blue/Green pods, traffic, history)
kubectl argo rollouts get rollout stock-pulse

# Watch rollout status in real-time (auto-refreshes)
kubectl argo rollouts get rollout stock-pulse --watch

# Promote: switch all traffic from old (Blue) to new (Green)
kubectl argo rollouts promote stock-pulse

# Rollback: instantly revert traffic to the previous stable version
kubectl argo rollouts undo stock-pulse

# Abort: stop a rollout in progress
kubectl argo rollouts abort stock-pulse
```

How to Access Preview (New Version) Before Promoting

```
# Forward the preview service to port 3001 on your laptop
kubectl port-forward svc/stock-pulse-preview 3001:3000
# Then open http://localhost:3001 to test the new version privately
```

Rollout Services

Service	Who uses it	Points to
stock-pulse-active	Real users / production	Blue pods (stable)
stock-pulse-preview	You / testers	Green pods (new version)

🔒 NGINX Ingress — Rate Limiting

Concepts

- Rate limiting prevents one IP address from overwhelming your server.
- Requests exceeding the limit get an **HTTP 503** response instantly.
- The limit is enforced by the NGINX Ingress controller **before** the request reaches your pods.

Ingress Annotations Reference

```
# In helm/stock-pulse/values.yaml under ingress.annotations:

# Max requests per SECOND per IP (very strict, for APIs)
nginx.ingress.kubernetes.io/limit-rps: "10"

# Max requests per MINUTE per IP (looser, for web pages)
nginx.ingress.kubernetes.io/limit-rpm: "200"

# How much burst is allowed above the limit before blocking
nginx.ingress.kubernetes.io/limit-burst-multiplier: "3"
```

Testing Rate Limiting

```
# Run the rate limit test (sends 100 requests rapidly)
sh test-rate-limit.sh

# Expected output:
# Request 1: 🟢 SUCCESS (200 OK)
# Request 2: 🟢 SUCCESS (200 OK)
# Request 3: 🟡 RATE LIMITED (503) ← blocked by NGINX!
```

⚠️ Critical: GitOps means changes **MUST** go through Git!

Editing `values.yaml` locally does nothing until you push to GitHub. ArgoCD reads from GitHub, not your hard drive.

```
# Correct way to apply an ingress annotation change:
git add helm/stock-pulse/values.yaml
git commit -m "chore: update rate limit to 20 rpm"
git push origin main
# Wait for ArgoCD to sync (or click Refresh in ArgoCD UI)
```

⚖️ Load Balancing

Concepts

- Kubernetes load balances automatically when `replicaCount > 1`.
- **kube-proxy** is the Kubernetes component that distributes packets across pods.
- Default algorithm: **Random** (approximates round-robin over time).
- Load balancing **ONLY** works through a **ClusterIP Service**, not through `port-forward`.

Scaling Pods

```
# In values.yaml, change:
replicaCount: 5 # Kubernetes will run 5 identical pods

# Then push to GitHub – ArgoCD applies the change automatically
```

Testing Load Balancing

```
# □ CORRECT WAY: Runs a temporary pod inside the cluster
# From inside, it hits the ClusterIP Service which properly round-robins
sh test-load-balance.sh

# □ WRONG WAY: port-forward goes to ONE fixed pod only
kubectl port-forward svc/stock-pulse-active 3000:3000
# All requests from port-forward go to the same pod – not a real LB test!
```

The whoami Endpoint

The file `app/api/whoami/route.ts` was created specifically for load balance testing. It returns the pod's hostname (which is the pod's name in Kubernetes):

```
{"pod":"stock-pulse-5447f954f9-bkpqv","timestamp":"2026-06-27T12:25:52.203Z"}
```

By calling this endpoint many times, you can see which pod handled each request.

Real Test Results (100 requests, 5 pods)

Final Distribution		
• stock-pulse-5447f954f9-bkpqv		27 reqs (27%)
• stock-pulse-5447f954f9-zd6xc		24 reqs (24%)
• stock-pulse-5447f954f9-gcdbs		18 reqs (18%)
• stock-pulse-5447f954f9-g8t47		18 reqs (18%)
• stock-pulse-5447f954f9-8gpd6		13 reqs (13%)

All 5 pods handled traffic. The distribution isn't perfectly equal because Kubernetes uses random selection, but it approximates 20% each over time.

□ Daily Startup Commands

Every time you restart your computer, run this single script:

```
sh start-system.sh
```

This script does:

1. Starts Minikube
2. Installs Argo Rollouts if missing
3. Waits for pods to be ready
4. Port-forwards everything in the background

What becomes accessible: | URL | What it opens | |---|---| `http://localhost:3000` | Live StockPulse app (active/blue pods) | `http://localhost:3001` | Preview StockPulse app

(green/preview pods) | | <https://localhost:8080> | ArgoCD Dashboard (admin / see password below) |

Get ArgoCD password:

```
kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath="{.data.password}" | base64 -d; echo
```

□ Git Workflow & Common Fixes

Standard Deploy Workflow

```
# 1. Make your code changes
# 2. Commit
git add .
git commit -m "feat: your description here"

# 3. Push (triggers GitHub Actions automatically)
git push origin main
```

Fix: "non-fast-forward" push rejection

This happens because GitHub Actions automatically committed the new image tag to your repo. Your local branch is behind.

```
# Pull the automated commit first, then push
git pull --rebase origin main
git push origin main
```

Permanent fix — Create a git alias

```
# Run once to set up
git config --global pull.rebase true
git config --global alias.sync '!git pull origin main && git push origin main'

# From now on, just use:
git sync
```

Fix: Large file rejected by GitHub

```
# Remove the file from staging
git reset HEAD~1
rm <large-file-name>
echo "<large-file-name>" >> .gitignore
git add .
git commit -m "fix: remove large file"
git push origin main
```

☐ Servers & Ports Reference

Component	Technology	Internal Port	Access Command	Local URL
StockPulse App	Node.js / Next.js	3000	<code>kubectl port-forward svc/stock-pulse-active 3000:3000</code>	<code>http://localhost:3000</code>
Preview App	Node.js / Next.js	3000	<code>kubectl port-forward svc/stock-pulse-preview 3001:3000</code>	<code>http://localhost:3001</code>
ArgoCD UI	Go (gRPC/REST)	443	<code>kubectl port-forward svc/argocd-server -n argocd 8080:443</code>	<code>https://localhost:8080</code>
Kubernetes API	Minikube	8443	(auto-configured by kubectl)	—
Yahoo Finance	External API	443	(called by your app server)	—
GitHub	External	443	(called by ArgoCD + Actions)	—
Docker Hub	External	443	(images pushed here by Actions)	—

🔍 Troubleshooting Guide

Error	Cause	Fix
<code>connect: no route to host</code>	Minikube is stopped	<code>minikube start</code>
<code>ImagePullBackOff</code>	Docker image doesn't exist on Docker Hub yet	Wait for GitHub Actions to finish building and pushing
<code>revision main must be resolved</code>	ArgoCD lost internet connection	<code>kubectl delete pods -n argocd -l app.kubernetes.io/name=argocd-repo-server</code>
YAML parse error on <code>deployment.yaml</code>	Commented-out YAML still has <code>{{ }}</code> Helm tags	Delete the file entirely, don't comment it out

Error	Cause	Fix
failed to push... non-fast-forward	GitHub Actions committed to your repo while you were working	<code>git pull --rebase origin main && git push origin main</code>
remote rejected: Large files detected	A binary file was accidentally committed	<code>git reset HEAD~1 && rm <file> && echo '<file>' >> .gitignore</code>
403 Permission denied to github-actions	Actions bot has read-only permission	GitHub Repo → Settings → Actions → General → Read & Write Permissions
All requests go to one pod	You're using <code>port-forward</code> which is not a load balancer	Use <code>sh test-load-balance.sh</code> which runs inside the cluster
Rate limit test shows all 200 OK	Rate limiter is not active (maybe not synced)	Push your <code>values.yaml</code> change to GitHub and wait for ArgoCD

□ Key Lessons Learned

1. **kubectl port-forward is NOT a load balancer.** It creates a single tunnel to one pod. For real load balancing tests, run requests from INSIDE the cluster using `kubectl run`.
2. **GitOps means Git is the only control panel.** Editing files locally and expecting them to deploy is wrong. Push to GitHub → ArgoCD sees it → applies it.
3. **[skip ci] prevents infinite loops.** When your pipeline commits back to the repo, it adds `[skip ci]` to the commit message to stop triggering another build.
4. **Commenting out YAML templates breaks Helm.** If a template file has `{{ .Values.xxx }}` tags inside YAML comments (`#`), Helm still processes the tags. Delete the file instead of commenting it out.
5. **kubectl port-forward svc/argocd-server -n argocd 8080:443** — Always specify namespace `-n argocd` and map `8080:443` (not `8080:80`). ArgoCD only listens on 443.
6. **Rate limiting is enforced at the Ingress layer** before requests reach your pods. Even with 5 pods, a rate-limited request never reaches any pod.
7. **Blue-Green with autoPromotionEnabled: false** keeps the old version alive until YOU click "Promote". This gives you a safe window to test the new version privately via the preview service.
8. **Minikube loses internet after sleep/network changes.** Restarting with `minikube stop && minikube start` fixes DNS and network connectivity for both ArgoCD (GitHub) and your app (Yahoo Finance).

